3) Linear Methods for Regression

Assume we have a dataset with with 3 predictors (x_1, x_2, x_3) and 1 dependent variable y:

	x_1	x_2	x_3	y
1 sample	1	2	1	3
1 predictor	3	1	5	2
	7	1	2	3

We write the input vector as:

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 1 & 5 \\ 7 & 1 & 2 \end{bmatrix}$$ Matrix Form

Similarly, the output vector is $Y = \begin{bmatrix} 3 \\ 2 \\ 3 \end{bmatrix}$

If we have p predictors and n datasamples, then, X has dimension $n \times p$ (n rows and p columns). Y has dimension $n \times 1$.

3.2) Linear Regression And Least Squares

The linear regression model, has form

$$Y = X\beta + \varepsilon \iff \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}_{n\times 1} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ & & \vdots & & \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}_{n\times k} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}_{k\times 1} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}_{n\times 1}$$

For a single observation, this becomes:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

The linear regression model has high bias as it assumes that $\mathbb{E}[Y|X]$ is linear (or assuming linearity is reasonable).

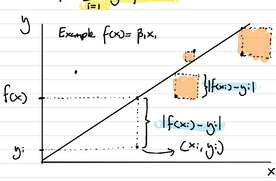
To estimate $\beta = [\beta_0, \beta_1, \ldots, \beta_p]^T$ we use a set of training data with N observations $(x_1, y_1), \ldots, (x_N, y_N)$:

- $x_i \in \mathbb{R}^p$, $x_i = [x_{i1}, x_{i2}, \ldots, x_{ip}]$
- $y_i \in \mathbb{R}$

Let f be our linear model: $f: \mathbb{R}^p \to \mathbb{R}$, $x_i \mapsto y_i$.

Then, the least squares method minimizes the residual sum of squares:

$$RSS(\beta) = \sum_{i=1}^{N} (y_i - f(x_i))^2$$



In matrix form:

$$RSS(\beta) = (Y - X\beta)^T (Y - X\beta)$$

The minimum is obtained by setting:

$$\frac{\partial RSS}{\partial \beta} = 0 \implies -2X^TY + 2X^TX\beta = 0 \implies X^TX\beta = X^TY$$

Worth giving this a deeper look

$$\text{OLS Solution} \implies \boxed{\hat{\beta} = (X^TX)^{-1}X^TY}$$

*) Assuming that X has full column rank, hence X^TX is invertible and positive definite.

$$\hat{Y} = X\hat{\beta} \qquad ^\wedge \text{ denotes an estimate}$$

$$= X(X^TX)^{-1}X^TY \qquad \text{(From OLS)}$$

$$= HY \qquad \text{(Calling } H = X(X^TX)^{-1}X^T\text{)}$$

H is sometimes called the hat matrix. Do you see why?

If the columns of X are not linearly independent (e.g. $x_1 = 2x_2$), then β is not uniquely defined. In this case, the OLS formula does not hold. This also happens if $p > N$.

The variance of the estimator is:

$$Var(\hat{\beta}) = (X^TX)^{-1}\sigma^2, \qquad \hat{\sigma}^2 = \frac{1}{N-p-1}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$\hat{\sigma}^2$ is an unbiased estimate of σ^2. If we assume the conditional expectation of Y ($\mathbb{E}[Y|X]$) is linear on $X_1, \ldots, X_p$, and Gaussian errors, then:

$$Y = \mathbb{E}(Y|X_1, \ldots, X_p) + \varepsilon$$

$$= \beta_0 + \sum_{j=1}^{p} X_j\beta_j + \varepsilon \qquad \varepsilon \sim N(0, \sigma^2)$$

Then, $\hat{\beta} \sim N(\beta, (X^TX)^{-1}\sigma^2)$ ⇒ This can be used to construct hypothesis tests and confidence intervals for β_j

The test hypothesis for a particular coefficient $\beta_j = 0$.

$$\text{Z-score:} \quad z_j = \frac{\hat{\beta}_j}{\hat{\sigma}\sqrt{v_j}}$$

v_j: j-th diagonal element of $(X^TX)^{-1}$

z_j is distributed as t_{N-p-1}. If σ is known (not estimated), then z_j is normally distributed. For $N - p - 1 > 100$ $t_{N-p-1} \approx N$.

F-statistic

$$F = \frac{\frac{RSS_0 - RSS_1}{p_1 - p_0}}{\frac{RSS_1}{n - p_1}}$$

RSS_1: bigger model
RSS_0: smaller model
p_1, p_0: # of predictor (β_0 counts)
N: # of training examples

Example

Baseline (model 0): $F(X) = \beta_0 + \beta_1 x_1 + \cdots + \beta_4 x_4$

- $RSS_0 = 32.81$ (Residual sum of squares on data)

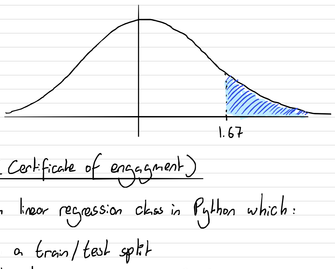
New (model 1): $F(X) = \beta_0 + \beta_1 x_1 + \cdots + \beta_8 x_8$ (add 4 predictors)

- $RSS_1 = 29.43$

If both models are trained on 67 data examples, then:

$$F = \frac{(32.81 - 29.43)/(9-5)}{29.43/(67-9)} = 1.67$$

$P(F_{4,58} > 1.67) = 0.17$ ⇒ Not significant



Exercises (Certificate of engagement)

1) Create a linear regression class in Python which:
   - Creates a train/test split
   - Computes linear regression via OLS
   - Computes the residual sum of squares of the model
   - Computes the hat matrix
   - Computes the variance of the estimate $\hat{\beta}$
   - Computes the z-scores of $\hat{\beta}$
2) Create a class to compute the F-statistic between two linear regression models.
3) Use it in the Kaggle playground competition.

3.3) SUBSET SELECTION

- Prediction accuracy
- Interpretation

Objective: Select a subset of $\hat{\beta} = (\beta_0, \beta_1, \dots, \beta_p)$

3.3.1) BEST-SUBSET SELECTION

- Find the subset of size k ($k \in \{0, 1, \dots, p\}$) that gives the smallest residual sum of squares:

$$RSS = \sum_{i=1}^{n} (y_i - X\beta)^2$$

$\dim(X) = n \times k$
$\dim(\beta) = k$

- Computationally expensive. Why?

How many ways are there of selecting k elements from p elements?

$$\binom{p}{k} = \frac{p!}{k!(p-k)!}$$

$n! = n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$

→ 'p choose k'

3.3.2) FORWARD- AND BACKWARD-STEPWISE SELECTION

- Sequentially add into the model the predictor that most improves the fit. (Forward)
- Sequentially subtract from the model the predictor that most preserves the fit. (Backward). Works by dropping the lowest z-score.

* Computationally more efficient, greedy algorithm, usually similar performance to best subset.

3.4) SHRINKAGE METHODS

Continuous version of subset selection.

3.4.1) RIDGE REGRESSION

- Shrinks coefficients by imposing a penalty on their size.

$$\hat{\beta}_R = \underset{\beta}{\operatorname{argmin}} \left(\sum_{i=1}^{N} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right)$$

$\beta = (\beta_1, \beta_2, \dots, \beta_p)$ - Exclude β_0

λ: Shrinkage parameter

Equivalently,

$$\hat{\beta}_R = \underset{\beta}{\operatorname{argmin}} \left(\sum_{i=1}^{N} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2 \right), \quad \sum_{j=1}^{p} \beta_j^2 < t$$

*) There is a 1-1 correspondance between t & λ.

We normalise the inputs. Can you tell why? How do we exclude β_0? Two ways:

1) Remove the column of 1s from X and do:

$$x_j = \frac{x_j - \mu_{x_j}}{\sigma_{x_j}}, \qquad y = y - \mu_y$$

$$\mu_{x_j} = \frac{1}{N} \sum_{i=1}^{N} x_{ij}, \qquad \sigma^2_{x_j} = \frac{1}{N} \sum_{i=1} (x_{ij} - \mu_{x_j})^2$$
$$\mu_y = \frac{1}{N} \sum_{i=1}^{N} y_i$$

Then $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$ (I has dim $p \times p$)

$\hat{\beta}_0 = \mu_y - \mu_x^T \hat{\beta}$ (μ_x has dim p)

2) Change I for $D = \begin{bmatrix} 0 & 0 & \dots \\ 0 & 1 & 0 \\ \vdots & 0 & 1 \end{bmatrix}$, then

$\hat{\beta} = (X^T X + \lambda D)^{-1} X^T y$ → β_0 included

3.4.2) THE LASSO

$$\hat{\beta}_L = \underset{\beta}{\operatorname{argmin}} \left[\sum_{i=1}^{N} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

- No analitical solution. gradient descent?

Exercises (Certificate of engagment)

1) Improve the LinearRegression class from Session 1 to include:
   - Best subset selection
   - Forward & BACKWARD-Stepwise selection
   - Ridge regression
2) Try your class methods on the three datasets:
   - What results do you obtain
   - Which one works best? Why?
3) Implemel the solution for the Lasso using gradient descent (Hard).

Regression by successive orthogonalisation

Recap

Linear regression: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$

In matrix form $Y = X\beta + \varepsilon$

We estimate $\beta = [\beta_0\ \beta_1 \dots \beta_p]^T$ by $\hat{\beta} = [\hat{\beta}_0\ \hat{\beta}_1 \dots \hat{\beta}_p]$

minimising $RSS(\beta) = \sum_{i=1}^{N} (y_i - \hat{y})^2 \quad \left(+ \lambda \sum_{j=1}^{p} \beta_j^2\right)$

Setting $\frac{\partial RSS}{\partial \beta} = 0 \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T y$

$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$ (Ridge)

Regression by successive orthogonalisation

Model: $y_i = \beta_0 + \sum_{i=1}^{p} \beta_i x_{ij} + \varepsilon_i$

Goal: compute $\hat{\beta} = [\beta_0\ \beta_1 \dots \beta_p]^T$

Step 1| For each predictor

compute: $\bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij}$ & $\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$

and we center the data. For every observation:

$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ } Similar to normalising

$\tilde{y}_i = y_i - \bar{y}$

The regression task becomes $\tilde{y} = \sum_{i=1}^{p} \beta_p \tilde{x}_i + \varepsilon$ → No intercept! Can you see why?

Step 2| Core idea

Two predictors are orthogonal if $x_j^T x_k = 0 \ (k \neq j)$.

All predictors are orthogonal if $x_j^T x_k = 0 \ \forall k, j$ with $k \neq j$

If that is the case, then $\beta_j = \frac{x_j^T y}{x_j^T x_j}$

Idea: build new orthogonal vectors $z_1, \dots, z_p$

Step 3| Core algorithm (Gram-Schmidt)

1) Set $z_1 = \tilde{x}_1$

2) Compute z_2: → Remove from $\tilde{x}_2$ the part explained by z_1

$$a_{12} = \frac{z_1^T \tilde{x}_2}{z_1^T z_1}, \quad z^2 = \tilde{x}_2 - a_{12} z_1$$

You can check $z_1^T z_2 = 0$ → Orthogonal

3) For z_3 (and z_j)

$$z^3 = \tilde{x}_3 - a_{13} z_1 - a_{23} z_2, \quad a_{kj} = \frac{z_k^T \tilde{x}_j}{z_k^T z_k}$$

Generally,

$$z_j = \tilde{x}_j - a_{1j} z_1 - a_{2j} z_2 - \dots - a_{(j-1)j} z_{(j-1)}$$

Pseudocode:

```
For j in 1..p:
    z_j = x_j
    for k in 1...j-1:
        a_kj = dot(z_k, x_j) / dot(z_k, z_k)
        z_j = z_j - a_kj z_k
```

Step 4| Now:

$\tilde{y} = \sum_{i=1}^{p} \hat{\gamma}_j z_j \quad \hat{\gamma}_j = \frac{z_j^T y}{z_j^T z_j} \quad (\hat{\beta} \neq \hat{\gamma})$

We can get predictions, but how do we obtain the original $\hat{\beta}$?

Step 5| In matrix form, we have:

$\tilde{X} = ZR \qquad R = \begin{bmatrix} 1 & a_{12} & a_{13} & \dots \\ 0 & 1 & a_{23} & \dots \\ 0 & 0 & 1 & \dots \\ \vdots & & & \ddots \end{bmatrix}$

and $\gamma = R\beta \Rightarrow \beta = R^{-1}\gamma$

Back substitution: → Since R is upper triangular

$\hat{\beta}_p = \gamma_p$ (Start with last predictor)

$$\hat{\beta}_j = \gamma_j - \sum_{k=j+1}^{p} R_{jk} \hat{\beta}_k$$

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^{p} \hat{\beta}_j \bar{x}_j$$

□

Why bother?

- Numerically stable, faster, standard in numerical libraries.
- Statistical interpretation of OLS

Exercises

1) Implement functions to center a vector and center the columns of a matrix.

2) Implement Gram-Schmidt

Input: X
Output: Z, R } Verify $X \approx Z @ R$

3) Compute the $\hat{\gamma}$ coefficients

4) Obtain $\hat{\beta}$

5) Create a full OLS via orthogonalisation function and compare your results with $\hat{\beta} = (X^T X)^{-1} X^T y$ (Vanilla OLS)